

```

    for(y = 0; y < n; y++) printf("%3d", a[x][y]);
    printf("\n");
}
return 0;
}

```

这段程序充分利用了 C 语言简洁的优势。首先,赋值 $x=0$ 和 $y=n-1$ 后马上要把它们作为数组 a 的下标,因此可以合并完成; tot 和 $a[0][n-1]$ 都要赋值 1,也可以合并完成。这样,就用一条语句完成了多件事情,而且并没有牺牲程序的可读性——这段代码的含义显而易见。

提示 3-6: 可以利用 C 语言简洁的语法,但前提是保持代码的可读性。

那 4 条 `while` 语句有些难懂,不过十分相似,因此只需介绍其中的第一条:不断向下走,并且填数。我们的原则是:先判断,再移动,而不是走一步以后发现越界了再退回来。这样,则需要进行“预判”,即是否越界,以及如果继续往下走会不会到达一个已经填过的格子。越界只需判断 $x+1 < n$,因为 y 的值并没有修改;下一个格子是 $(x+1, y)$,因此只需“ $a[x+1][y] == 0$ ”,简写成“ $!a[x+1][y]$ ”(其中“ $!$ ”是“逻辑非”运算符)。

提示 3-7: 在很多情况下,最好是在做一件事之前检查是不是可以做,而不要做完再后悔。因为“悔棋”往往比较麻烦。

细心的读者也许会发现这里的一个“潜在 bug”:如果越界, $x+1$ 会等于 n , $a[x+1][y]$ 将访问非法内存!幸运的是,这样的担心是不必要的。“ $\&\&$ ”是短路运算符(还记得我们在那里提到过吗?)。如果 $x+1 < n$ 为假,将不会计算“ $!a[x+1][y]$ ”,也就不会越界了。

至于为什么是 $++tot$ 而不是 $tot++$,留给读者思考。

3.2 字符数组

文本处理在计算机应用中占有重要地位。本书到现在为止还没有正式讨论过字符串(尽管曾经使用过),因为在 C 语言中,字符串其实就是字符数组——可以像处理普通数组一样处理字符串,只需要注意输入输出和字符串函数的使用。

竖式问题。找出所有形如 $abc*de$ (三位数乘以两位数)的算式,使得在完整的竖式中,所有数字都属于一个特定的数字集合。输入数字集合(相邻数字之间没有空格),输出所有竖式。每个竖式前应有编号,之后应有一个空行。最后输出解的总数。具体格式见样例输出(为了便于观察,竖式中的空格改用小数点显示,但所写程序中应该输出空格,而非小数点)。

样例输入:

2357

样例输出：

```
<1>
..775
X..33
-----
.2325
2325.
-----
25575
```

The number of solutions = 1

【分析】

本题的思路应该是很清晰的：尝试所有的 abc 和 de ，判断是否满足条件。我们可以写出整个程序的伪代码：

```
char s[20];
int count = 0;
scanf("%s", s);
for(int abc = 111; abc <= 999; abc++)
    for(int de = 11; de <= 99; de++)
        if("abc*de"是个合法的竖式)
        {
            printf("<%d>\n", count);
            打印 abc*de 的竖式和其后的空行
            count++;
        }
printf("The number of solutions = %d\n", count);
```

第一个新内容是 `char s[20]` 定义。`char` 是“字符型”的意思，而字符是一种特殊的整数。为什么字符会是特殊的整数？请参见图 3-1 所示的 ASCII 编码表。

0	NUL	1	SOH	2	STX	3	ETX	4	EOT	5	ENQ	6	ACK	7	BEL
8	BS	9	HT	10	NL	11	VT	12	NP	13	CR	14	SO	15	SI
16	DLE	17	DC1	18	DC2	19	DC3	20	DC4	21	NAK	22	SYN	23	ETB
24	CAN	25	EM	26	SUB	27	ESC	28	FS	29	GS	30	RS	31	US
32	SP	33	!	34	"	35	#	36	\$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	/	93	]	94	-	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124	—	125	}	126	~	127	DEL

图 3-1 ASCII 编码表

从图 3-1 中可见，每一个字符都有一个整数编码，称为 ASCII 码。为了方便书写，C 语

言允许用直接的方法表示字符，例如，“a”代表的就是a的ASCII码。不过，有一些字符直接表示出来并不方便，例如，回车符是“`\n`”，而空字符是“`\0`”，它也是C语言中字符串的结束标志。其他例子包括“`\\`”（注意必须有两个反斜线）、“`\"`”（这个是单引号），甚至还有的字符有两种写法：“`\"`”和“`\"`”都表示双引号。像这种以反斜线开头的字符称为转义序列（Escape Sequence）。如果认真完成了第1章中的实验，相信对这些字符不会陌生。

提示 3-8：C语言中的字符型用关键字 `char` 表示，它实际存储的是字符的ASCII码。字符常量可以用单引号法表示。在语法上可以把字符当作 `int` 型使用。

另一个新内容是“`scanf("%s", s)`”。和“`scanf("%d", &n)`”类似，它会读入一个不含空格、TAB和回车符的字符串，存入字符数组 `s`。注意，不是“`scanf("%s", &s)`”，`s`前面没有“`&`”符号。

提示 3-9：在“`scanf("%s", s)`”中，不要在 `s` 前面加上“`&`”符号。如果是字符串数组 `char s[maxn][maxl]`，可以用“`scanf("%s", s[i])`”读取第 `i` 个字符串。注意，“`scanf("%s", s)`”遇到空白字符会停下来。

接下来有两个问题：判断和输出。根据我们的一贯作风，先考虑输出，因为它比较简单。每个竖式需要打印7行，但不一定要用7条 `printf` 语句，1条足矣。首先计算第一行乘积 $x=abc*e$ ，然后是第二行 $y=abc*d$ ，最后是总乘积 $z=abc*de$ ，然后一次性打印出来：

```
printf("%5d\nX%4d\n-----\n%5d\n%4d\n-----\n%5d\n\n", abc, de, x, y, z);
```

注意这里的 `%5d`，它表示按照5位数打印，不足5位在前面补空格（还记得 `%03d` 吗？）。完整程序如下：

程序 3-4 竖式问题

```
#include<stdio.h>
#include<string.h>
int main()
{
    int count = 0;
    char s[20], buf[99];
    scanf("%s", s);
    for(int abc = 111; abc <= 999; abc++)
        for(int de = 11; de <= 99; de++)
        {
            int x = abc*(de%10), y = abc*(de/10), z = abc*de;
            sprintf(buf, "%d%d%d%d", abc, de, x, y, z);
            int ok = 1;
            for(int i = 0; i < strlen(buf); i++)
                if(strchr(s, buf[i]) == NULL) ok = 0;
            if(ok)
```

```

    {
        printf("<%d>\n", ++count);
        printf("%5d\nX%4d\n-----\n%5d\n%4d\n-----\n%5d\n\n", abc, de, x, y, z);
    }
}

printf("The number of solutions = %d\n", count);
return 0;
}

```

还有两个函数是以前没有遇到的: `sprintf` 和 `strchr`。 `strchr` 的作用是在一个字符串中查找单个字符, 而这个 `sprintf` 似曾相识: 之前用过 `printf` 和 `fprintf`。没错! 这 3 个函数是“亲兄弟”, `printf` 输出到屏幕, `fprintf` 输出到文件, 而 `sprintf` 输出到字符串。多数情况下, 屏幕总是可以输出的, 文件一般也能写 (除非磁盘满或者硬件损坏), 但字符串就不一定了: 应该保证写入的字符串有足够的空间。

提示 3-10: 可以用 `sprintf` 把信息输出到字符串, 用法和 `printf`、`fprintf` 类似。但应当保证字符串足够大, 可以容纳输出信息。

多大才算足够大呢? 答案是字符个数加 1, 因为 C 语言的字符串是以空字符 “\0” 结尾的。后面还会提到这个问题, 但是基本原则仍然是以前说过的: 如果算不清楚就把数组空间设置得大一点, 空间够用的情况下浪费一点没关系。例如, 此处声明的缓冲字符串 `buf` 的长度为 99 (可以保存长度为 98 的字符串), 保存 `abc, de, x, y, z` 的所有数字绰绰有余。

函数 `strlen(s)` 的作用是获取字符串 `s` 的实际长度。什么叫实际长度呢? 字符数组 `s` 的大小是 20, 但并不是所有空间都用上了。如果输入是 “2357”, 那么实际上 `s` 只保存了 5 个字符 (不要忘记了还有一个结束标记 “\0”), 后面 15 个字符是不确定的 (还记得吗? 变量在赋值之前是不确定的)。`strlen(s)` 返回的就是结束标记之前的字符个数。因此这个字符串中的各个字符依次是 `s[0], s[1], ..., s[strlen(s)-1]`, 而 `s[strlen(s)]` 正是结束标记 “\0”。

提示 3-11: C 语言中的字符串是以 “\0” 结尾的字符数组, 可以用 `strlen(s)` 返回字符串 `s` 中结束标记之前的字符个数。字符串中的各个字符是 `s[0], s[1], ..., s[strlen(s)-1]`。

提示 3-12: 由于字符串的本质是数组, 它也不是 “一等公民”, 只能用 `strcpy(a, b)`, `strcmp(a, b)`, `strcat(a, b)` 来执行 “赋值”、“比较” 和 “连接” 操作, 而不能用 “=”、“==”、“<=”、“+” 等运算符。上述函数都在 `string.h` 中声明。

此处再次看到了 `++count` 这样的用法, 有必要对它进行进一步说明。猜猜看: `count=0` 时, “`printf("%d %d %d\n", count++, count++, count++)`” 会输出什么 (然后做个实验)。怎么样, 是不是和你想的不同呢?

另一个例子是 “`count = count++`”。这里对 `count++` 的解释是: `count++` 在表达式中的值是加 1 之前的值 (即原来的值), 但计算 `count++` 之后 `count` 会增加 1。问题出现了: 这个 “稍后再加 1” 到底是何时进行的呢? 如果是计算完赋值的右边 (即 `count++`) 之后